PANDADAW DOCUMENTACION

Jorge Morgado, Cristian Ortega y Ángel Sánchez

PandaDaw - Documentación Técnica Completa

Versión: 1.0
Fecha: Febrero 2026
Equipo de Desarrollo: Jorge morgado, cristian ortega y angel sanchez

Tarifa: 25€/hora/desarrollador

Índice General

Nota: Esta documentación está diseñada para facilitar la conversión a Word. Para cada sección se incluyen diagramas Mermaid que se renderizan automáticamente en GitHub/Markdown y pueden exportarse a imágenes.

1. Información del Proyecto

- 1.1 Proyectos en la Solución
- 1.2 Características Implementadas

2. Arquitectura del Sistema

- 2.1 Diagrama de Arquitectura General
- 2.2 Patrón de Arquitectura (Clean Architecture)
- 2.3 Tecnologías Utilizadas

3. Estructura del Proyecto

- 3.1 Organización de Carpetas
- 3.2 Dependencias entre Proyectos

4. Modelo de Datos

- 4.1 Entidades y Relaciones (Diagrama ER)
- 4.2 Detalle de Entidades
- 4.3 Enumeraciones
- 4.4 Restricciones y Validaciones

5. Catálogo de Productos (DataSeeder)

- 5.1 Categorías Disponibles
- 5.2 Productos de Ejemplo

6. API REST Endpoints

- 6.1 Autenticación (Auth)
- 6.2 Carrito
- 6.3 Productos

- 6.4 Valoraciones
- 6.5 Ventas

7. Servicios Backend

- 7.1 Interfaces de Servicios
- 7.2 Validadores

8. Páginas de la Aplicación

- 8.1 Ruta de cada Página
- 8.2 Permisos Requeridos

9. Componentes Blazor (Tiempo Real)

- 9.1 NotificacionesToast
- 9.2 ValoracionesRealtime
- 9.3 ContadorCarrito
- 9.4 SignalR Hub

10. Tecnologías y Dependencias

- 10.1 Paquetes NuGet
- 10.2 Herramientas de Desarrollo

11. Flujos de Usuario y Casos de Uso

- 11.1 Flujo de Registro e Inicio de Sesión (Secuencia)
- 11.2 Flujo de Compra Completo (Secuencia)
- 11.3 Flujo de Valoraciones en Tiempo Real
- 11.4 Casos de Uso Detallados

12. Presupuesto del Proyecto

- 12.1 Desglose de Costes
- 12.2 Estimación de Horas

13. Mejores Prácticas Implementadas

- 13.1 Patrones de Diseño
- 13.2 Testing
- 13.3 Seguridad

14. Glosario

- 14.1 Términos Técnicos

15. Anexos

- 15.1 Configuración de Desarrollo

- 15.2 Despliegue

16. Próximos Pasos

- 16.1 Mejoras Futuras
- 16.2 Expansiones Planificadas

---

1. Resumen Ejecutivo

PandaDaw es una plataforma de comercio electrónico desarrollada con tecnología **.NET 10** que permite la venta de productos de electrónica de consumo. El proyecto está formado por una solución con 4 proyectos:

1.1 Proyectos en la Solución

Proyecto	Tipo	Descripción
PandaDawRazor	Web Application	Interfaz de usuario con Razor Pages y Blazor Server
PandaBack	Class Library	API REST, servicios, repositorios y modelos
Tests	Unit Tests	Pruebas unitarias con xUnit
PandaDaw-Playwright	E2E Tests	Pruebas end-to-end con Playwright

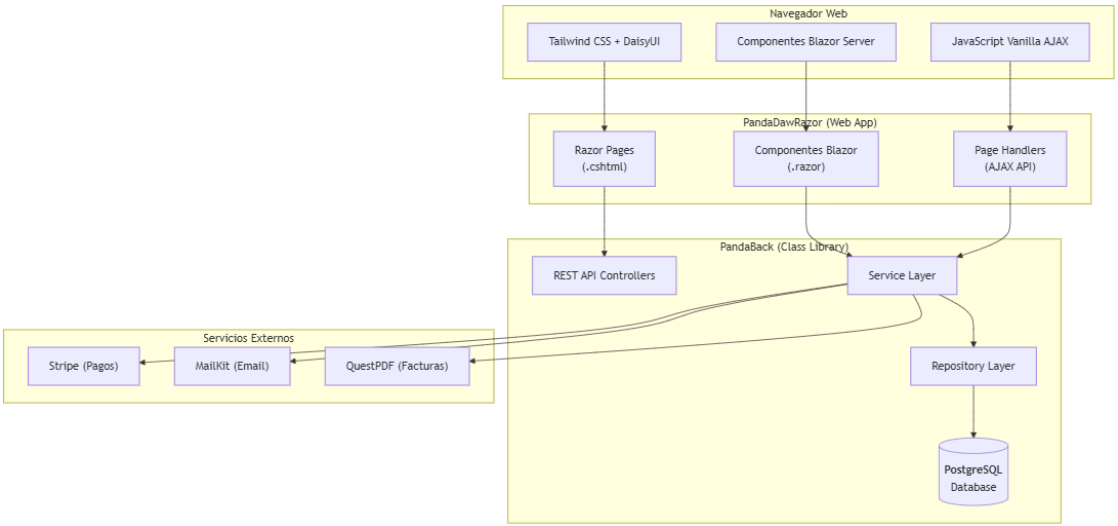
1.2 Características Implementadas

- Carrito de compras persistente
- Sistema de favoritos por usuario
- Sistema de valoraciones y reseñas en tiempo real
- Autenticación con ASP.NET Identity
- Autorización basada en roles (User/Admin)
- Gestión completa de productos (CRUD)
- Sistema de pagos con Stripe
- Generación de facturas con QuestPDF
- Envío de emails con MailKit
- Notificaciones en tiempo real (Blazor/SignalR)
- Diseño responsivo con Tailwind CSS + DaisyUI
- Base de datos PostgreSQL con Entity Framework Core

---

2. Arquitectura del Sistema

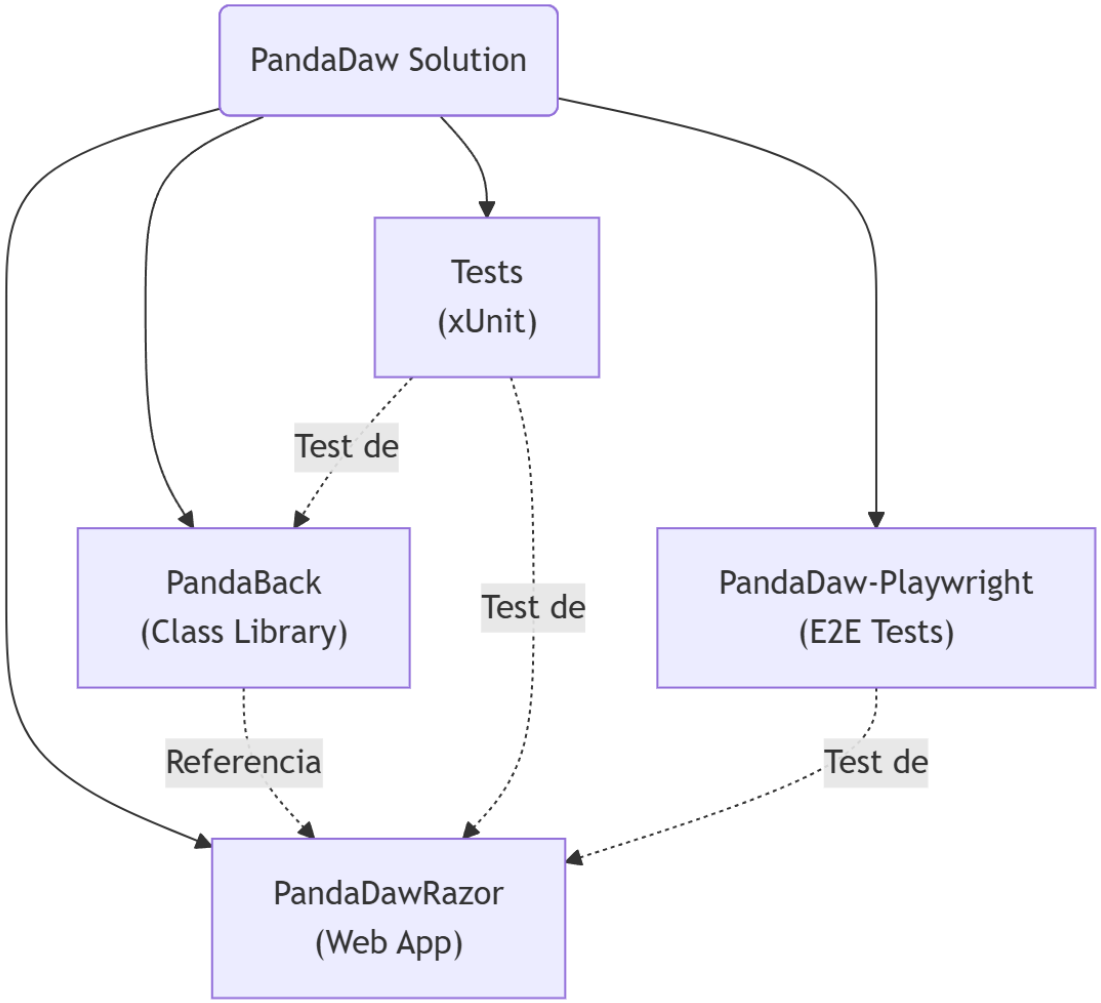
2.1 Diagrama de Arquitectura General



2.2 Patrón de Arquitectura (Clean Architecture)

3. Estructura del Proyecto

3.1 Proyectos y Dependencias



🔗3.2 Estructura de Carpetas - PandaBack

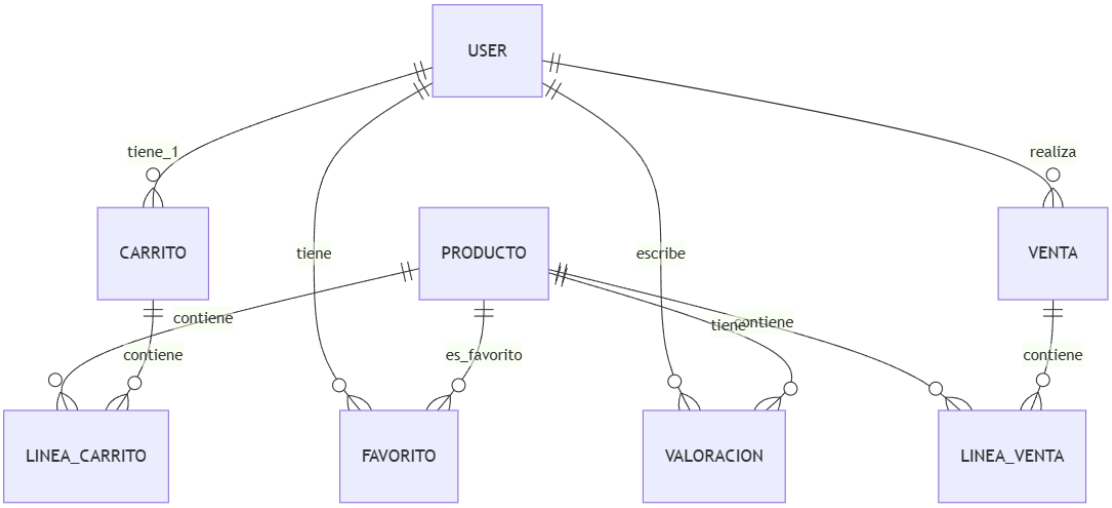
```
PandaBack/
├── config/                      # Configuraciones
│   ├── CacheConfig.cs          # Configuración de caché
│   └── DotEnvLoader.cs         # Carga de variables de entorno
├── Data/                       # Capa de datos
│   ├── PandaDbContext.cs       # Contexto EF Core
│   └── DataSeeder.cs           # Datos iniciales
├── Dtos/                       # Objetos de Transferencia
│   ├── Auth/                   # DTOs de autenticación
│   ├── Carrito/               # DTOs del carrito
│   ├── Favoritos/              # DTOs de favoritos
│   ├── Productos/              # DTOs de productos
│   ├── Valoraciones/           # DTOs de valoraciones
│   └── Ventas/                 # DTOs de ventas
├── Errors/                     # Manejo de errores
│   └── PandaError.cs          # Errores tipados
├── Mappers/                    # Mapeo DTO ↔ Modelo
├── Middleware/                 # Middleware personalizado
│   └── GlobalExceptionHandler.cs
├── Models/                     # Entidades del dominio
│   ├── Carrito.cs
│   ├── Categoria.cs
│   ├── Favorito.cs
│   ├── LineaCarrito.cs
│   ├── LineaVenta.cs
│   ├── Producto.cs
│   ├── User.cs
│   ├── Valoracion.cs
│   ├── Venta.cs
│   ├── Role.cs
│   └── EstadoPedido.cs
├── Repositories/               # Repositorios (datos)
├── RestController/             # Controladores API REST
├── Services/                   # Servicios (lógica)
│   ├── Auth/
│   ├── Carrito/
│   ├── Email/
│   ├── Factura/
│   ├── Favoritos/
│   ├── Productos/
│   ├── Stripe/
│   ├── Valoraciones/
│   └── Ventas/
├── Validators/                 # Validadores FluentValidation
└── Program.cs
```

3.3 Estructura de Carpetas - PandaDawRazor

```
PandaDawRazor/
├── Components/                  # Componentes Blazor
│   ├── ContadorCarrito.razor   # Badge contador carrito
│   ├── NotificacionesToast.razor # Notificaciones toast
│   └── ValoracionesRealtime.razor # Valoraciones en tiempo real
├── Filters/                    # Filtros de páginas
│   └── NavBadgePageFilter.cs
├── Pages/                      # Páginas Razor
│   ├── AdminPanel.cshtml       # Panel de administración
│   ├── Api.cshtml             # Endpoint AJAX
│   ├── Carrito.cshtml         # Carrito de compras
│   ├── Detalle.cshtml         # Detalle de producto
│   ├── Favoritos.cshtml        # Lista de favoritos
│   ├── Index.cshtml           # Página principal
│   ├── Login.cshtml           # Inicio de sesión
│   ├── Pago.cshtml            # Página de pago
│   ├── PagoExitoso.cshtml     # Confirmación de pago
│   ├── Pedidos.cshtml         # Historial de pedidos
│   ├── Register.cshtml        # Registro de usuario
│   ├── Logout.cshtml          # Cerrar sesión
│   ├── Error.cshtml           # Página de error
│   └── Shared/
│       └── _Layout.cshtml     # Layout principal
├── Services/                   # Servicios cliente
│   └── NotificacionService.cs # Servicio de notificaciones
├── wwwroot/                   # Archivos estáticos
│   ├── css/
│   ├── js/
│   └── lib/
├── Program.cs
└── PandaDawRazor.csproj
```

4. Modelo de Datos

4.1 Entidades y Relaciones



4.2 Detalle de Entidades

Usuario (User)

- **Herencia:** IdentityUser (ASP.NET Identity)
- **Campos propios:** Nombre, Apellidos, Avatar, Role, FechaAlta, IsDeleted
- **Relaciones:** Carrito (1:1), Ventas (1:N), Valoraciones (1:N), Favoritos (1:N)

Producto

- **Campos:** Id, Nombre, Descripcion, Precio, Stock, Imagen, Category, FechaAlta, IsDeleted
- **Índices:** Category, IsDeleted
- **Relaciones:** LineaCarrito (1:N), LineaVenta (1:N), Favoritos (1:N), Valoraciones (1:N)

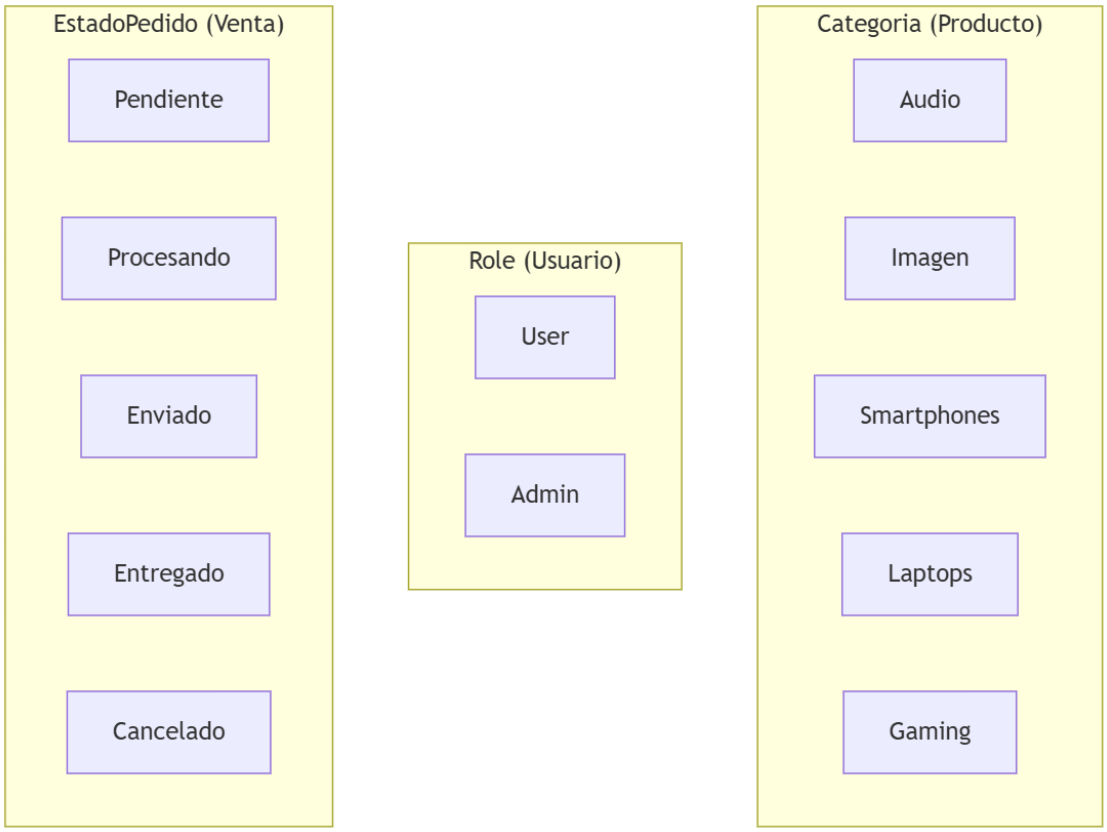
Carrito

- **Campos:** Id, UserId, FechaActualizacion
- **Relaciones:** Usuario (N:1), LineaCarrito (1:N)

Venta

- **Campos:** Id, UserId, Total, Estado, FechaVenta
- **Relaciones:** Usuario (N:1), LineaVenta (1:N)

4.3 Enumeraciones



5. Catálogo de Productos (DataSeeder)

El sistema incluye **20 productos** precargados en las siguientes categorías:

Categoría	Productos
Smartphones	iPhone 15 Pro Max, Samsung Galaxy S24 Ultra, Google Pixel 8 Pro, Xiaomi 14 Pro
Audio	AirPods Pro 2ª gen, Sony WH-1000XM5, Bose QuietComfort Ultra, JBL Flip 6
Laptops	MacBook Pro 16" M3 Max, ASUS ROG Zephyrus G14, Dell XPS 15, MacBook Air 15" M3
Gaming	PS5 Slim, Nintendo Switch OLED, Xbox Series X, Valve Steam Deck OLED
Imagen	Sony Alpha 7 IV, LG OLED 65" C3, DJI Mini 4 Pro, GoPro HERO12 Black

5.1 Usuarios de Prueba

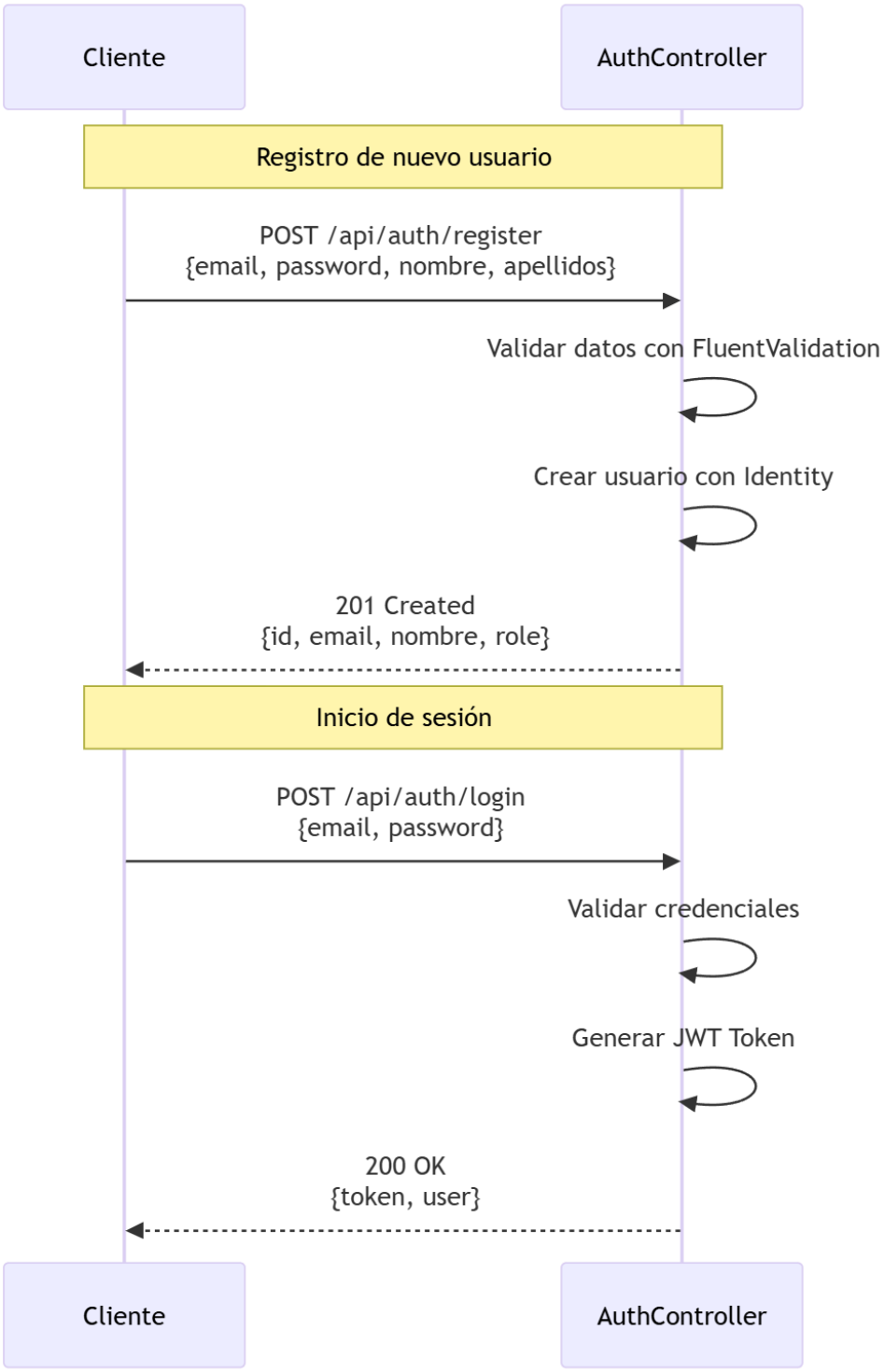
Email	Contraseña	Rol
admin@pandadaw.com	Admin123!	Admin
usuario1@pandadaw.com	Usuario123!	User
usuario2@pandadaw.com	Usuario123!	User

6. API REST Endpoints

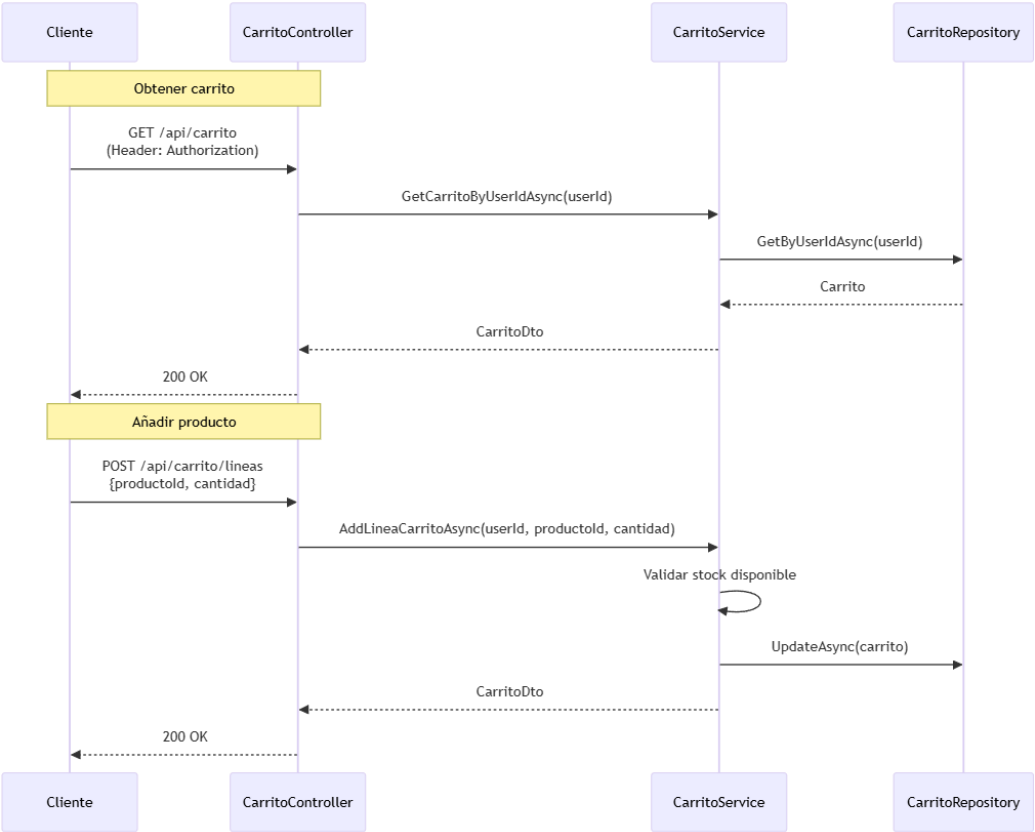
6.1 Controladores Disponibles

Controlador	Prefijo	Métodos	Autenticación
AuthController	/api/auth	POST register, POST login	Pública
ProductosController	/api/productos	GET, GET {id}, GET categoria, POST, PUT, DELETE	Admin para write
CarritoController	/api/carrito	GET, POST lineas, PUT lineas/{id}, DELETE lineas/{id}, DELETE	Requiere Auth
FavoritosController	/api/favoritos	GET {userId}, POST, DELETE {id}	Requiere Auth
ValoracionesController	/api/valoraciones	GET producto, POST, PUT {id}, DELETE {id}	Auth para write
VentasController	/api/ventas	GET, GET mis-pedidos, GET {id}, GET {id}/factura, POST, PATCH {id}/estado	Mixto

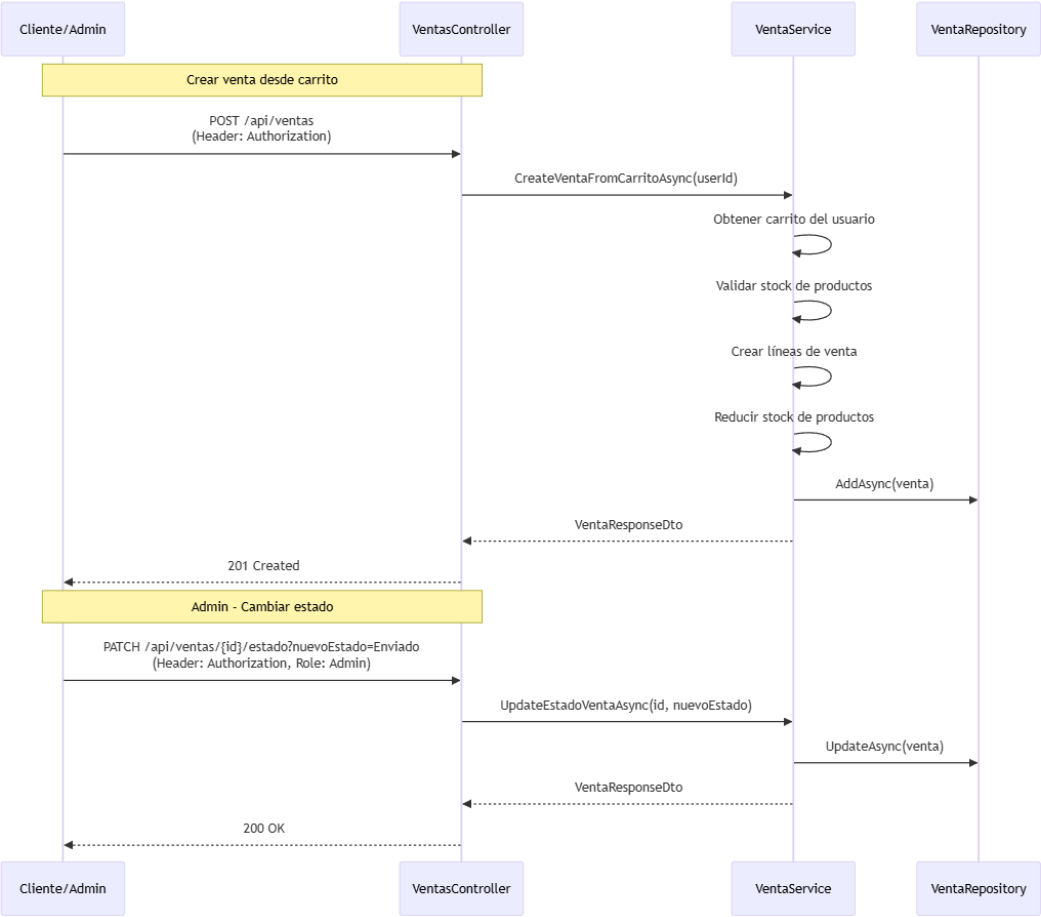
6.2 Diagrama de Endpoints - Auth



6.3 Diagrama de Endpoints - Carrito



6.4 Diagrama de Endpoints - Ventas



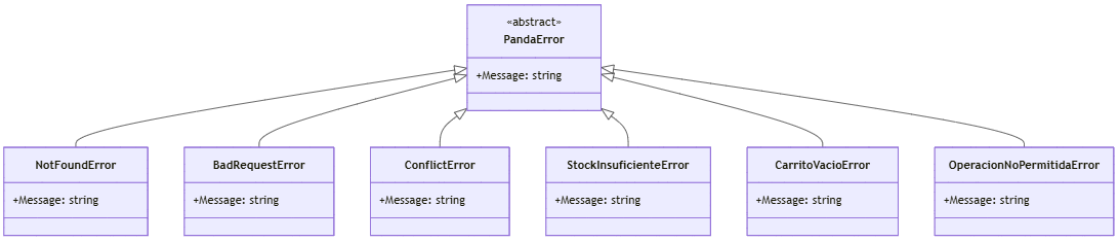
6.5 Tabla de Códigos de Respuesta

Código	Significado
200	OK - Operación exitosa
201	Created - Recurso creado
204	No Content - Operación exitosa sin contenido
400	Bad Request - Datos inválidos
401	Unauthorized - No autenticado
403	Forbidden - Sin permisos
404	Not Found - Recurso no encontrado
500	Internal Server Error - Error del servidor

7. Servicios Backend (Interfaces)

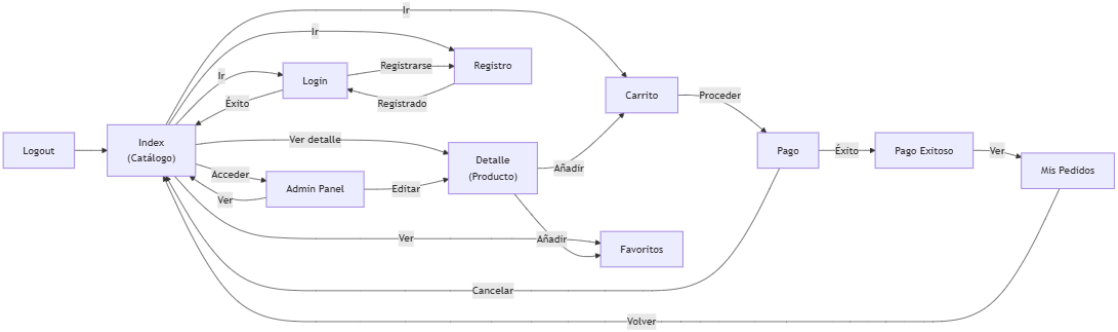
7.1 Servicios Implementados - Código Real

7.2 Errores del Sistema (Código Real)



8. Páginas de la Aplicación

8.1 Mapa de Navegación



8.2 Descripción de Páginas

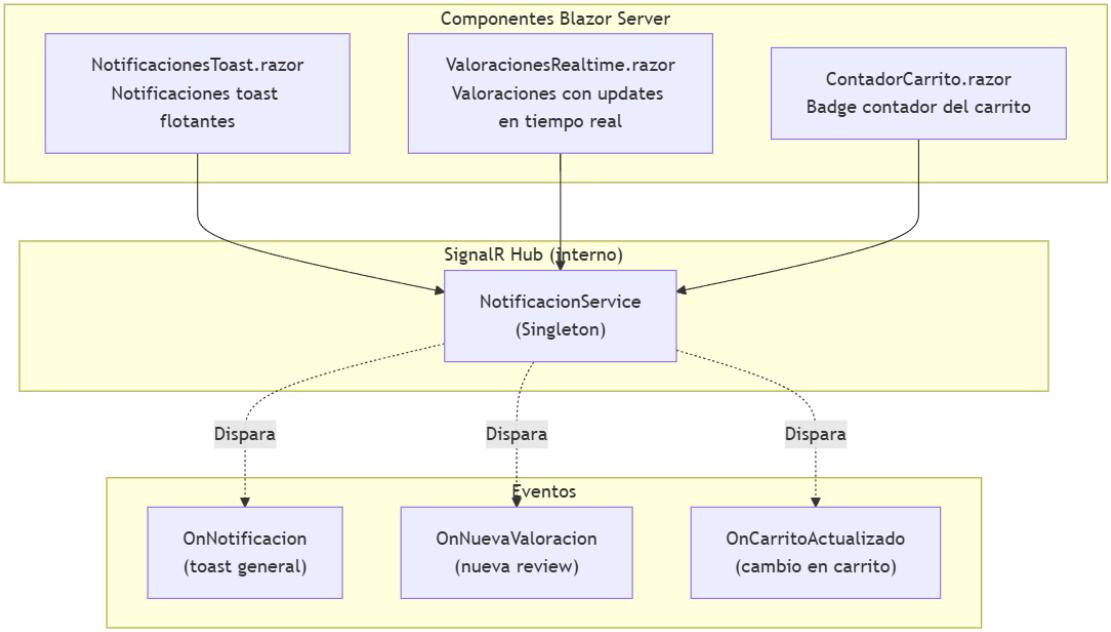
Página	Ruta	Descripción	Requiere Auth
Index	/	Catálogo principal con filtros por categoría	No
Detalle	/Detalle/{id}	Información completa de un producto	No
Carrito	/Carrito	Carrito de compras	Sí
Favoritos	/Favoritos	Lista de productos favoritos	Sí
Pago	/Pago	Proceso de checkout con Stripe	Sí
PagoExitoso	/PagoExitoso	Confirmación de compra	Sí
Pedidos	/Pedidos	Historial de pedidos	Sí
Login	/Login	Inicio de sesión	No

Página	Ruta	Descripción	Requiere Auth
Register	/Register	Registro de nuevo usuario	No
AdminPanel	/AdminPanel	Gestión de productos	Admin

---

9. Componentes Blazor (Tiempo Real)

9.1 Componentes Implementados



9.2 NotificacionService (Singleton)

```
// Eventos disponibles
public event Action<string, Notificacion>? OnNotificacion;      // Para u
n usuario
public event Action<string, Notificacion>? OnNotificacion;      // Para t
odos
public event Action<long>? OnNuevaValoracion;                    // Nueva
 valoración
public event Action<string, int>? OnCarritoActualizado;          // Carri
to actualizado
```

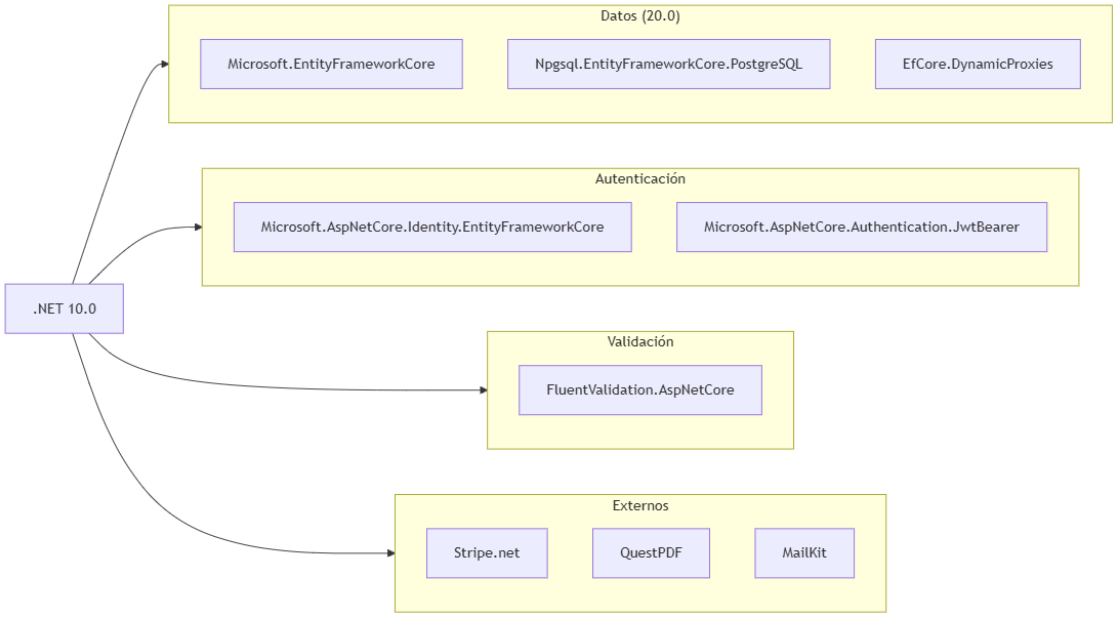
10. Tecnologías y Dependencias

10.1 Stack Tecnológico

Capa	Tecnología	Versión

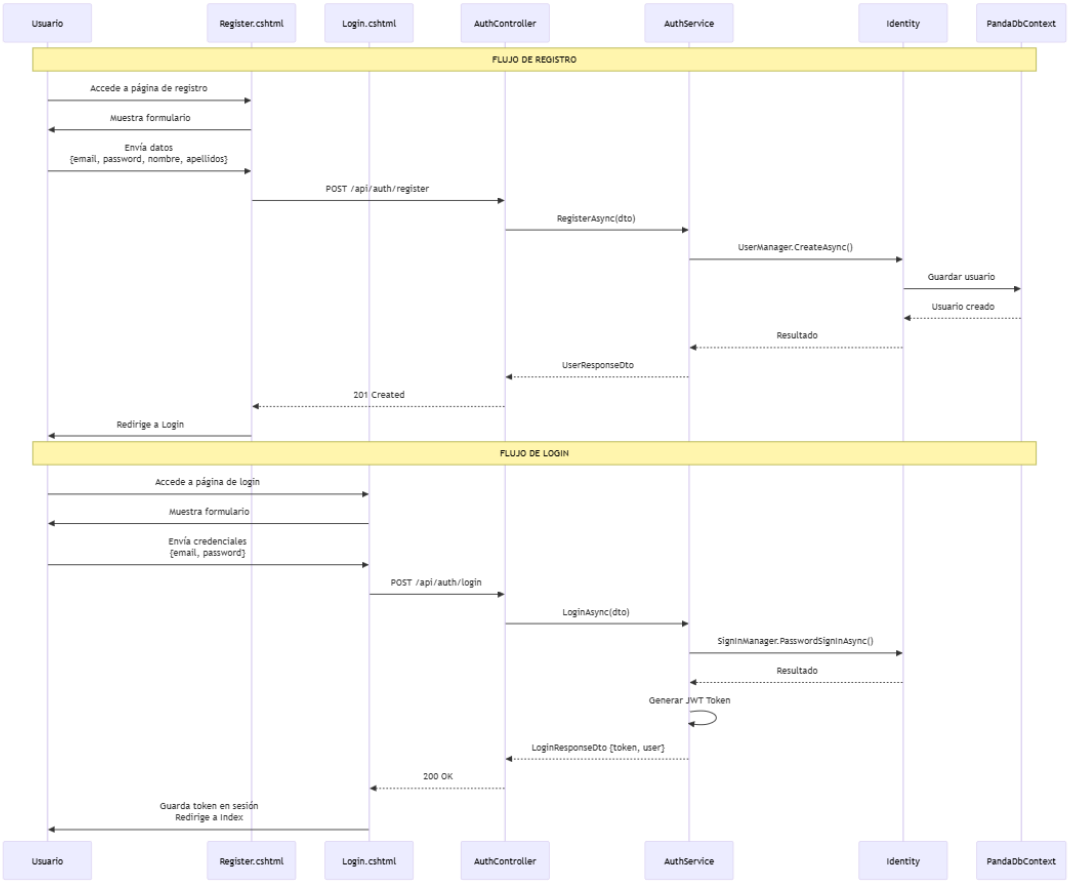
Capa	Tecnología	Versión
Runtime	.NET	10.0
Frontend	ASP.NET Core Razor Pages	10.0
Componentes	Blazor Server	10.0
CSS Framework	Tailwind CSS	3.x
UI Components	DaisyUI	4.x
Icons	Font Awesome	6.x
Database	PostgreSQL	-
ORM	Entity Framework Core	10.0
Auth	ASP.NET Identity	-
API	REST (ApiController)	-
Real-time	Blazor SignalR	-
Testing Unit	xUnit	-
Testing E2E	Playwright	-

10.2 Paquetes NuGet Principales

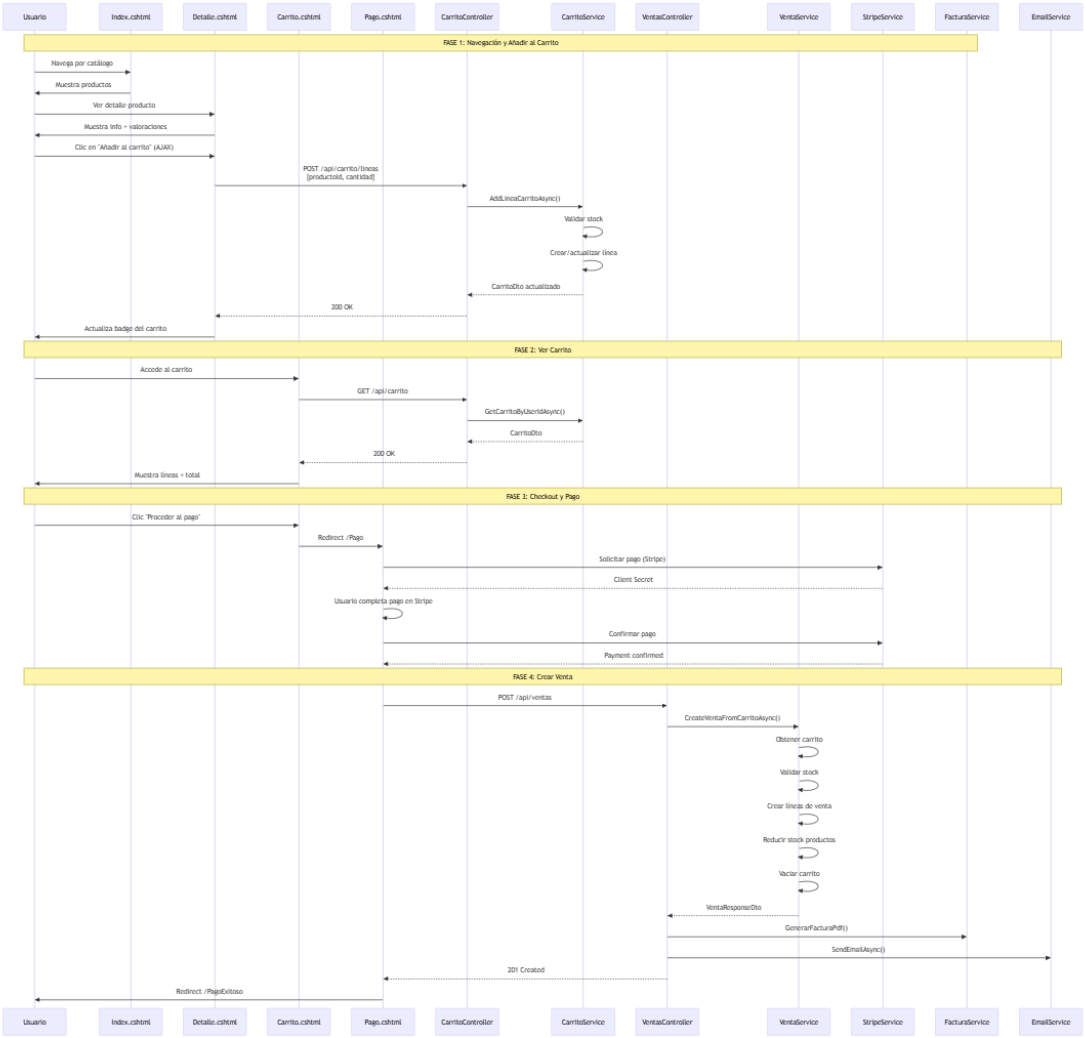


11. Flujos de Usuario

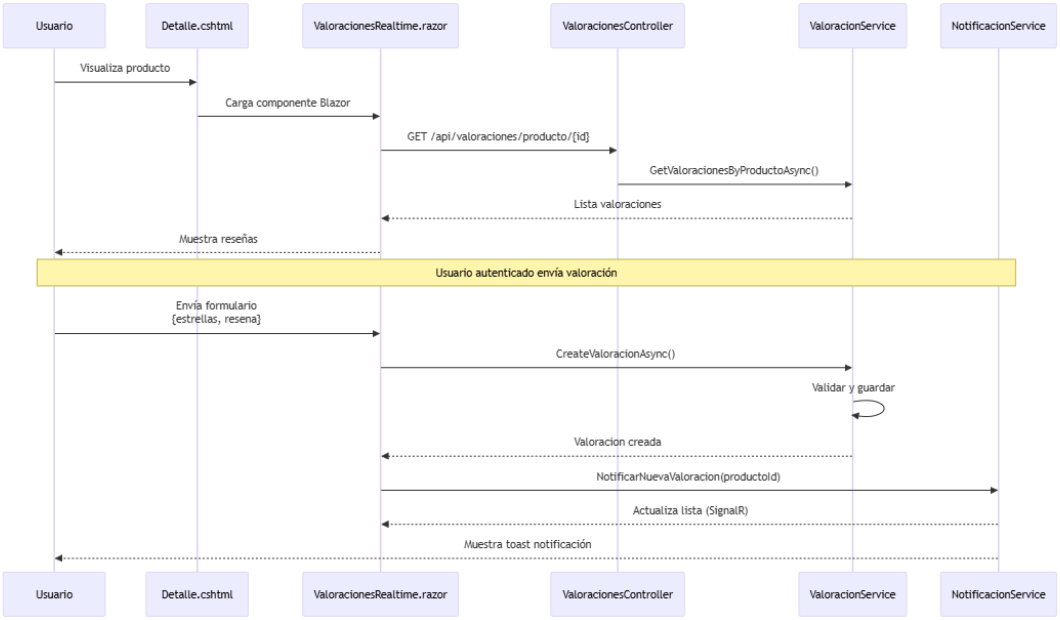
11.1 Flujo de Registro e Inicio de Sesión



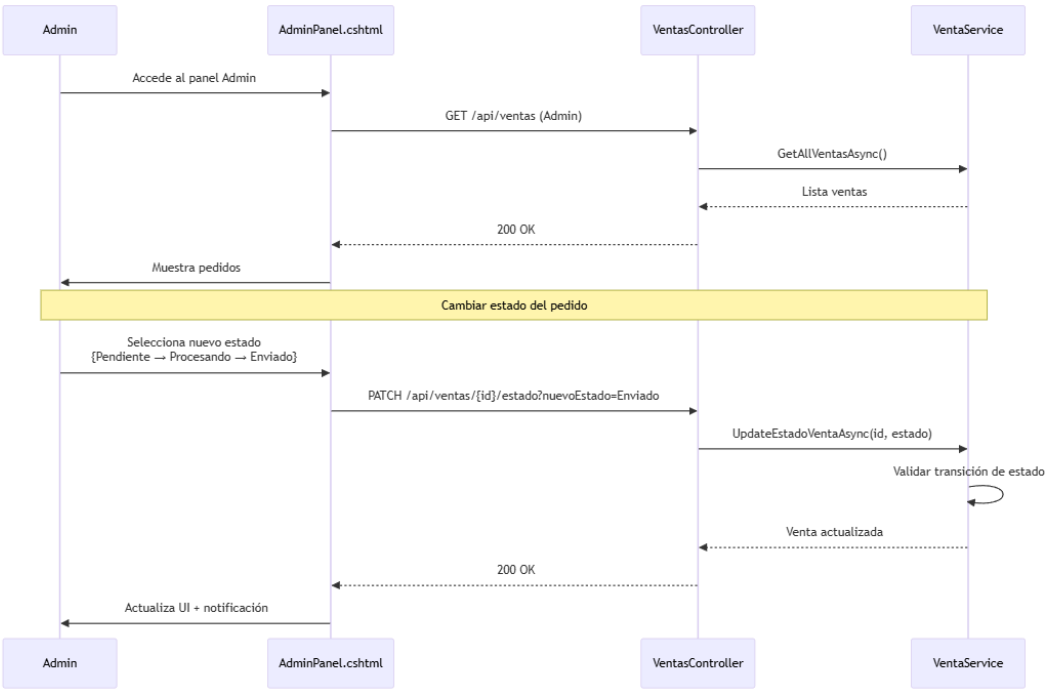
11.2 Flujo de Compra Completo



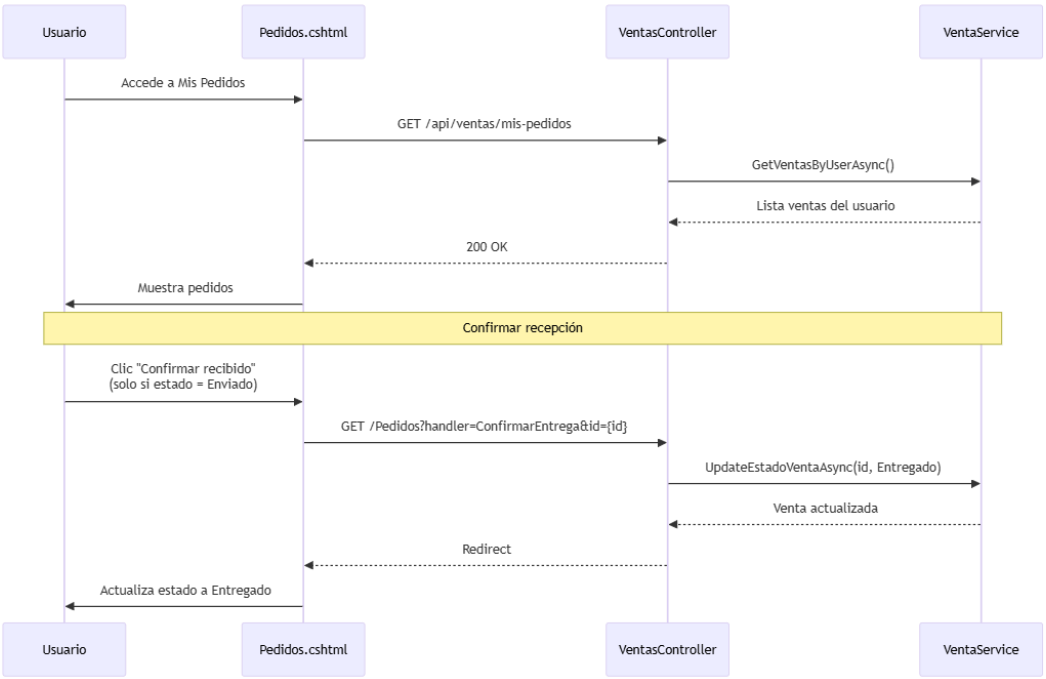
11.3 Flujo de Valoraciones



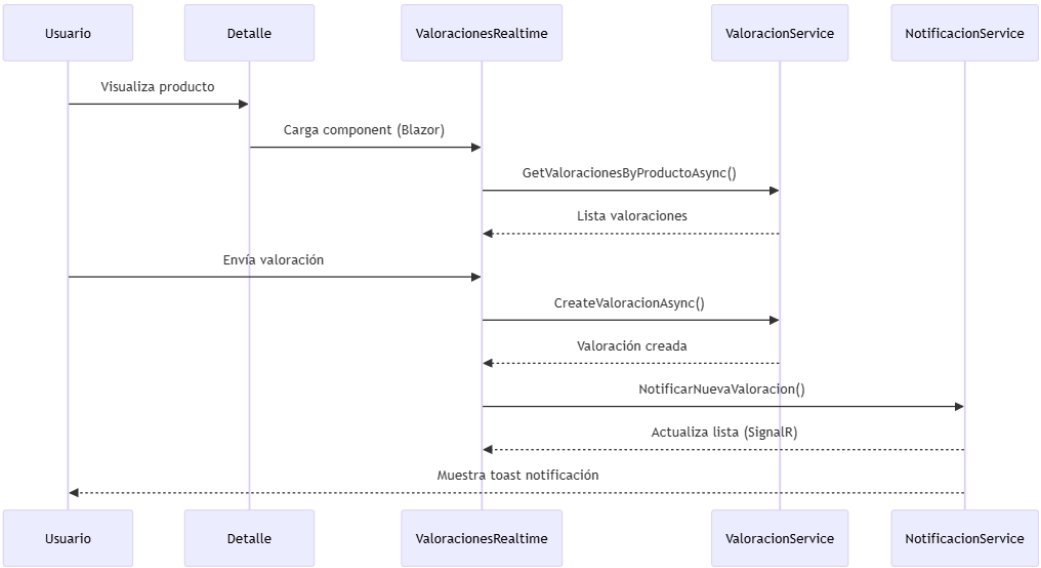
11.4 Flujo de Gestión de Pedidos (Admin)



11.5 Flujo de Confirmación de Entrega (Usuario)



11.2 Flujo de Valoración



11.5 Casos de Uso Detallados

ID	Caso de Uso	Actor	Precondición
UC-001	Registro de Usuario	Visitante	Ninguna
UC-002	Inicio de Sesión	Usuario Registrado	Usuario existente
UC-003	Ver Catálogo	Usuario	Ninguna

ID	Caso de Uso	Actor	Precondición
UC-004	Ver Detalle Producto	Usuario	Ninguna
UC-005	Añadir al Carrito	Usuario Auth	Sesión activa
UC-006	Gestionar Carrito	Usuario Auth	Carrito con items
UC-007	Proceso de Pago	Usuario Auth	Carrito con items
UC-008	Valorar Producto	Usuario Auth	Ha comprado producto
UC-009	Gestión Favoritos	Usuario Auth	Sesión activa
UC-010	Panel Admin	Admin	Rol Admin
UC-011	Notificaciones Real-time	Usuario Auth	Conexión SignalR

---

12. Presupuesto del Proyecto

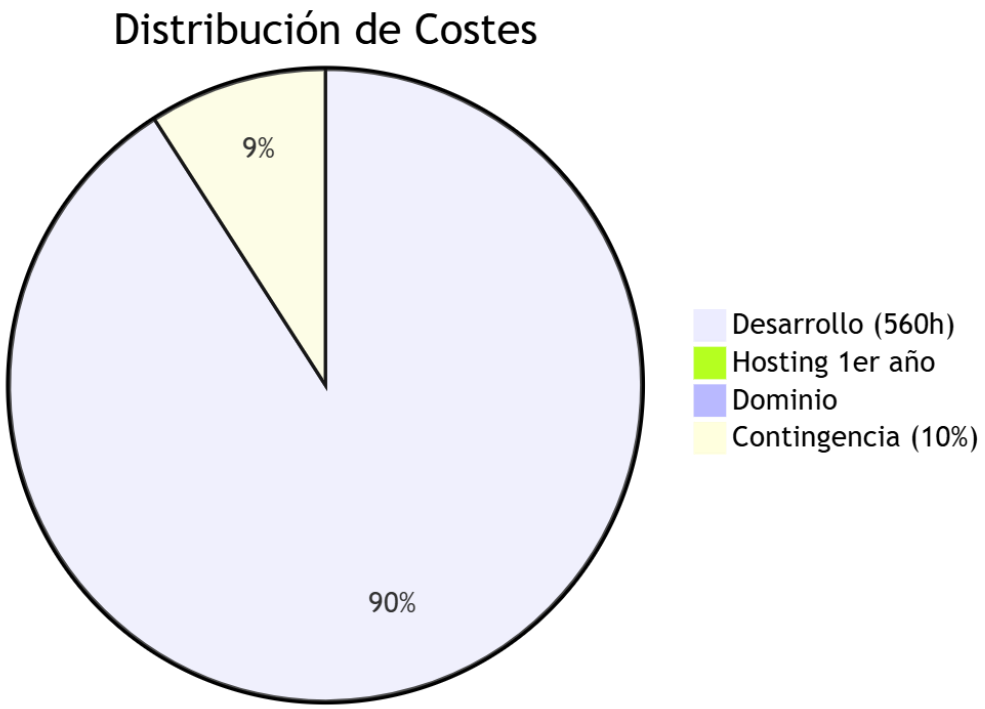
12.1 Estimación de Horas por Fase

Fase	Descripción	Horas	Coste (3 devs)
Fase 1: Setup	Configuración solución, proyectos, NuGet, Docker	24h	1.800€
Fase 2: Modelos y DTOs	Entidades EF Core, DTOs, Mapeadores	40h	3.000€
Fase 3: Repositorios	CRUD con EF Core, queries personalizadas	56h	4.200€
Fase 4: Servicios	Lógica de negocio, validaciones, errores	72h	5.400€
Fase 5: API REST	Controladores, documentación Swagger	32h	2.400€
Fase 6: Frontend Pages	Razor Pages principales (10 páginas)	80h	6.000€
Fase 7: Componentes Blazor	Notificaciones, valoraciones real-time	40h	3.000€
Fase 8: UI/UX	Tailwind, DaisyUI, animaciones, responsive	56h	4.200€
Fase 9: Integraciones	Stripe, QuestPDF, MailKit	24h	1.800€
Fase 10: Testing	Unit tests (xUnit), E2E (Playwright)	48h	3.600€
Fase 11: Documentación	README, comentarios XML, this doc	24h	1.800€
Fase 12: Ajustes y Tests	Bug fixes, optimizaciones, refinamiento	64h	4.800€
	TOTAL	**560h**	**42.000€**

12.2 Costes Adicionales (Año 1)

Concepto	Coste Estimado
Dominio (pandadaw.com)	12€/año
Hosting (Railway/Render - PostgreSQL)	25€/mes × 12 = 300€/año
Hosting (Railway/Render - Web App)	0-15€/mes × 12 = 0-180€/año
SSL (Let's Encrypt)	Gratis
Total Infra Year 1	**312-492€**

12.3 Resumen Budget



Concepto	Importe
Desarrollo (560h × 25€ × 3 devs)	42.000€
Infraestructura Year 1	~400€
Dominio	12€
Contingencia (10%)	4.200€
TOTAL PROYECTO	**~46.612€**

12.4 Distribución por Desarrollador

Métrica	Valor
Horas totales	560h

Métrica	Valor
Horas por desarrollador	~187h
Coste por desarrollador	14.000€

---

13. Mejores Prácticas Implementadas

13.1 Patrones de Diseño

- **Repository Pattern**: Abstracción del acceso a datos
- **Service Layer**: Lógica de negocio encapsulada
- **DTOs**: Transferencia de datos controlada
- **Dependency Injection**: Inyección de dependencias nativa
- **Singleton Pattern**: NotificacionService
- **Result Pattern**: CSharpFunctionalExtensions para manejo de errores

13.2 Seguridad

- **ASP.NET Identity**: Autenticación robusta
- **Password Hashing**: BCrypt con Identity
- **Role-based Authorization**: Admin vs Usuario
- **Input Validation**: FluentValidation
- **SQL Injection Protection**: Entity Framework parametrizado
- **AntiForgery Tokens**: Protección CSRF

13.3 Calidad de Código

- **XML Documentation**: Todos los métodos documentados
- **InheritDoc**: Interfaces implementadas coninheritdoc
- **Code Analysis**: nullable reference types
- **Async/Await**: Operaciones asíncronas

14. Glosario

Término	Definición
DTO	Data Transfer Object - Objeto para transferir datos entre capas
Repository	Patrón para abstraer el acceso a datos
Service	Clase que encapsula lógica de negocio
Blazor	Framework .NET para crear interfaces web interactivas
Razor Pages	Modelo de desarrollo web basado en páginas MVC
SignalR	Biblioteca para comunicación en tiempo real

Término	Definición
Identity	Sistema de autenticación y autorización de ASP.NET
FluentValidation	Biblioteca para validaciones fluent
Soft Delete	Eliminación lógica (IsDeleted = true)
Data Seeder	Datos iniciales cargados al crear la BD

15. Anexos

A. Comandos Útiles

```
# Restaurar paquetes
dotnet restore

# Compilar proyecto
dotnet build

# Ejecutar tests unitarios
dotnet test

# Ejecutar aplicación
dotnet run --project PandaDawRazor

# Añadir migración EF Core
dotnet ef migrations add InitialCreate --project PandaBack

# Actualizar base de datos
dotnet ef database update

# Ejecutar tests E2E
dotnet test PandaDaw-Playwright
```

B. Variables de Entorno (.env)

```
# Database
DB_CONNECTION_STRING=Host=localhost;Database=pandadaw;Username=postgres;P
assword=your_password

# JWT
JWT_SECRET=your-super-secret-key-min-32-chars
JWT_EXPIRY_HOURS=24

# Stripe
Stripe__PublishableKey=pk_test_...
Stripe__SecretKey=sk_test_...

# Email (SMTP)
SMTP_HOST=smtp.gmail.com
```

```
SMTP_PORT=587
SMTP_USER=your@email.com
SMTP_PASSWORD=your_password
```

C. Estructura de la Solución

```
PandaDaw/
├── PandaDaw.sln
├── PandaBack/
│   ├── PandaBack.csproj
│   └── [carpetas del proyecto]
├── PandaDawRazor/
│   ├── PandaDawRazor.csproj
│   └── [carpetas del proyecto]
├── Tests/
│   ├── Tests.csproj
│   └── Services/
└── PandaDaw-Playwright/
    ├── PandaDaw-Playwright.csproj
    └── Tests/
```